CSC1202 – Prompt Engineering / LLM agents
Week 1

Agenda

- **Computing in the Agentic AI Era**
- **Using agents; engineering agents**
- **Skills for the AI-Native Software Engineer**
- **Rethinking Software Engineering Education**

From Traditional ML to Large / Foundation Models

Traditional ML models

- Typically developed for specific tasks (e.g., image classification, spam detection, credit-risk prediction)
- Trained using task-specific data
- Different tasks often require different models

Large / Foundation Models

- Trained at scale on large and diverse datasets
- Provide broad capabilities across many tasks
- Can be adapted to different applications

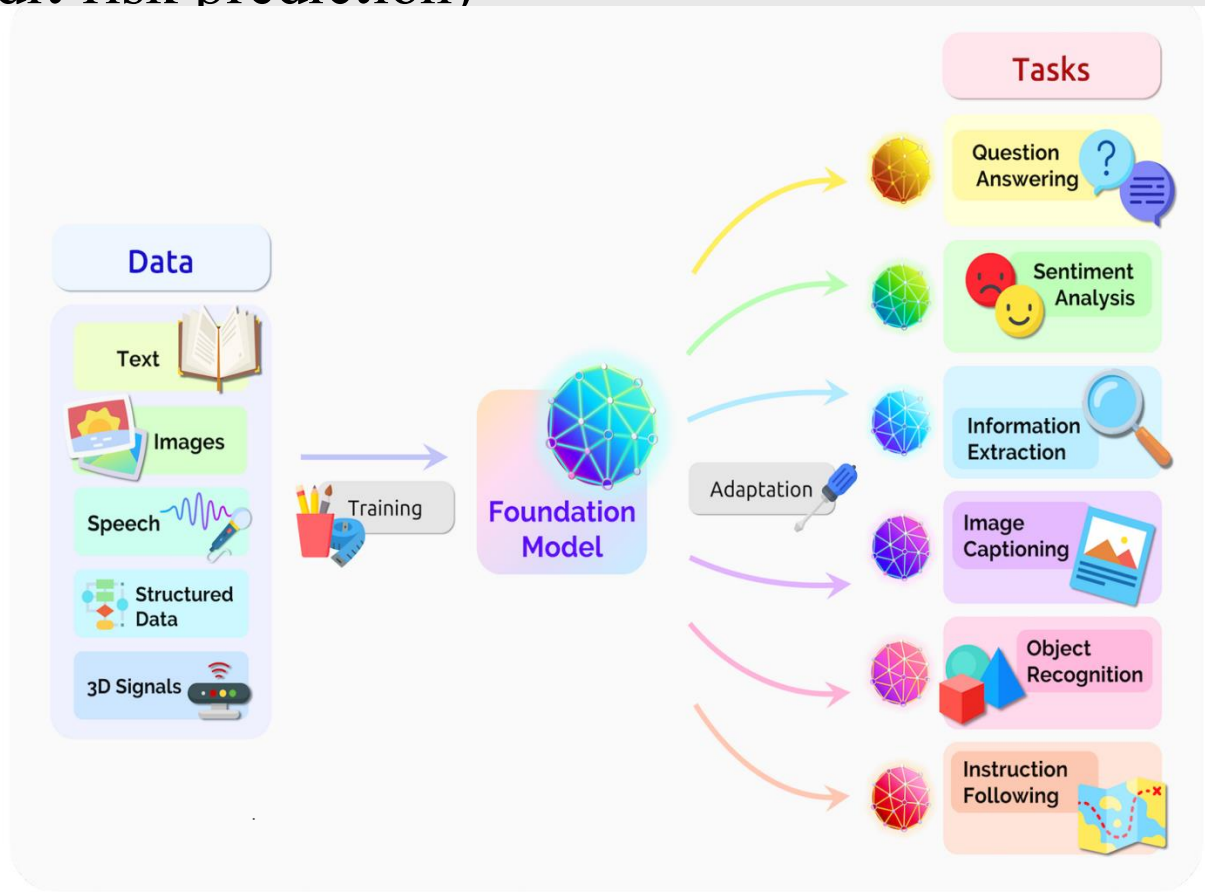


Fig. 2. A foundation model can centralize the information from all the data from various modalities. This one model can then be adapted to a wide range of downstream tasks.

Source: Bommasani et al., "On the Opportunities and Risks of Foundation Models," 2021.

Large / Foundation Models — Examples

Text / Language	GPT, Claude, Gemini, Llama, Qwen
Image	DALL·E, Stable Diffusion, Imagen
Video	Sora, Veo
Audio / Speech	Whisper, MusicGen
Multimodal	GPT-4o, Gemini 2.5, Claude 4, Llama 4

Examples span different modalities; some current models support more than one modality.

Large Language Models (LLMs)

- Process input and generate output as sequences of tokens
- Generate output by predicting the next token based on the input and previously generated tokens
- Tokens: A prompt is converted into small units called tokens (e.g., words or parts of words) before being processed by the LLM.
- Understand and generate natural language
- Can perform many tasks: question answering, summarisation, translation, code generation, etc.
- The same LLM can support many different tasks through natural-language instructions (prompts)
- Examples: GPT, Claude, Gemini, Llama, Qwen

Natural language provides a general interface for interacting with LLMs

What Makes LLMs Different from Traditional ML?

LLMs shift from task-specific models toward general-purpose models that can be instructed through natural language.

Traditional ML	LLMs
Typically designed for a specific task	General-purpose; can support many tasks
Often requires separate models/training for different tasks	The same model can perform different tasks
Inputs are typically defined for the specific task	Can interact through natural language
Primarily used for prediction or classification	Can understand and generate language and content
Behaviour largely determined during model development/training	Behaviour can be guided at use time through prompts and context

Quiz 1 (True / False statements)

1. Traditional ML models are typically developed and trained for specific tasks.
2. The same traditional ML model can generally perform many unrelated tasks simply by giving it different natural-language instructions.
3. LLMs can support different tasks, such as question answering, summarisation, translation, and code generation.
4. LLM behaviour can be guided at use time through natural-language prompts and context.
5. LLMs generate text by predicting the next token based on the input and previously generated tokens.
6. Using an LLM for a new task always requires training a new model for that task.
7. LLMs are examples of large/foundation models designed to provide broad capabilities rather than being developed for only one specific task.

Quiz 1 : Answers

1 True; 2 False; 3 True; 4 True; 5 True; 6 False; 7 True.

Adapting LLMs

The same LLM can be adapted to different tasks and applications.

Approach	What we do	Does the model change?
Prompting	Tell the LLM what we want using instructions and examples	No
Context / RAG	Give the LLM relevant information when it is needed	No
Fine-tuning	Further train the LLM using selected examples/data	Yes

- Prompting: change the instructions
- RAG / Context: change the information available
- Fine-tuning: change the model
- Other adaptation techniques exist

Powerful, But an LLM Alone Is Not Enough

Many real-world tasks require capabilities beyond generating a response.

External knowledge:	Access information not contained in the model
Up-to-date or private information:	Retrieve current or organisation-specific data
Multi-step reasoning and planning:	Break a complex task into steps
Memory and context:	Maintain relevant information across interactions
Tools and actions:	Search, call APIs, execute functions, interact with systems
Feedback and checking:	Observe results, verify them, and adapt, ….

LLM + additional capabilities → more capable AI systems

From LLMs to LLM-Based Agents

- An agent enables LLM capabilities to be used in an interactive process of deciding and acting toward a goal.

LLM

- Processes and generates information
- Can reason, plan, answer questions, generate code, etc.
- Operates on the context provided to it

LLM-Based Agent

- Uses an LLM as a core component
- Works toward a goal
- Can interact with tools and its environment
- Can use observations/results to determine subsequent actions

Quiz 2

Task: Label each example with the most appropriate approach/capability: Prompting (P), Context (C), RAG (R), Fine-tuning (F), Planning (PL), Memory (M), Tools/Actions (T)

1. Give the LLM instructions and examples showing how you want it to answer.

2. Provide the LLM with a document containing information needed to answer a question.

3. Retrieve relevant information from a collection of documents and provide it to the LLM.

4. Further train the LLM using selected examples/data.

5. Break a complex task into a sequence of steps.

6. Maintain relevant information across interactions.

7. Call an external API to retrieve information or perform an action.

Quiz 2: Answers

P, C, R, F, PL, M, T.

The New Programming Language Is Natural Language



Andrej Karpathy

@karpathy

The hottest new programming language is English

20:14 · 24 Jan 23

1.1M Views **1,259** Retweets **172** Quote Tweets

Source : https://twitter.com/karpathy/status/1617979122625712128?lang=en

AI Is Already Changing How Software Is Developed

Writing code manually Vs Developing software with assistance from LLM-powered coding tools

- **LLM-based tools are widely used by developers:** 84% use or plan to use AI tools; 51% of professional developers use them daily.
- **AI coding assistants:** GitHub Copilot, ChatGPT, … (like a chabot)
- **AI-native IDEs / vibe-coding platforms:** Cursor, Windsurf, Replit, Lovable,
- **CLI-based coding agents:** Developers increasingly work with AI agents directly from the terminal to inspect repositories, edit code, run tests and debug *(e.g., Claude Code, Codex CLI, Gemini CLI)*.
- **Vibe coding:** Describe tasks in natural language. AI generates/reviews/modifies… artifacts. Developer inspects and iterates.

Sources: Stack Overflow Developer Survey 2025; Karpathy, "Software Is Changing (Again)," 2025.

From AI Coding Assistants to Software Engineering Agents

Generate/suggest code **Vs** Perform software-engineering tasks

- **Coding assistants:** Help the developer while the developer drives the work *(e.g., GitHub Copilot code completion/chat)*.

- **Coding/SE agents:** Take a development task and perform multiple actions *(inspect files → plan → modify code → run tests → observe results → iterate)*.
  - **Examples:** *Cursor Agent, Claude Code, Codex, Devin, GitHub Copilot coding agent.*

- **Beyond coding:** Agents can support broader SE activities *(e.g., requirements, design, debugging, testing, review, maintenance)*.

- **Agentic behaviour:** Plan multiple steps, use tools, observe results and adapt what they do next (agent loop).

Key distinction: Assistant helps developer perform the task. **Agent** performs a sequence of actions toward completing the task (involving human collaboration).

What Is an AI Agent?

LLM: generates responses**. AI agent:** pursues goals and takes actions

- **Goal / Task:** Understand what the user (can be another agent) wants to achieve.
- **Use context & memory:** Access relevant information, knowledge and previous interactions.
- **Reason & plan:** Determine actions and their sequence.
- **Act:** Execute actions using tools/skills, such as APIs, applications, code execution and search.
- **Observe:** Inspect results from actions and the environment.
- **Adapt:** Use observations and feedback to determine subsequent actions.
- …..

Agent loop: Goal → Reason/Plan → Act → Observe → Adapt → …

Coding/Software Engineering Agents: Examples

- **Coding agents:** Generate, modify and refactor code, e.g., Claude Code, Codex, Cursor Agent.
- **Debugging agents:** Locate faults, analyse failures and propose/implement fixes.
- **Testing agents:** Generate tests, execute them and analyse failures.
- **Code-review agents:** Inspect changes for defects, security issues and maintainability problems.
- **Requirements/design agents:** Help analyse requirements and generate/design software artefacts.
- **End-to-end SE agents:** Work on issues across repository (e.g., exploration, planning, implementation, testing).

Key point: Agentic AI is expanding from code generation to multiple activities across the software lifecycle.

Business-process agents: Examples

- **Process modelling agents:** Analyse requirements and generate or modify process models *(e.g., BPMN models)*.
- **Process execution agents:** Perform process activities using applications, APIs and tools *(e.g., process an order or service request)*.
- **Process monitoring agents:** Monitor process execution, detect exceptions/delays and identify required interventions.
- **Process adaptation agents:** Respond to changes or failures by replanning, selecting alternatives or adapting process execution.

Examples of LLM Agents Across Application Domains

- **Research agents:** Search multiple sources, collect evidence, analyse information and synthesise findings *(e.g., literature or market research)*.
- **Customer-service agents:** Handle customer requests and take actions using organisational tools *(e.g., answer a query, check an order, process a return)*.
- **Data-analysis agents:** Access datasets, perform analyses and interpret results *(e.g., analyse sales data and generate insights)*.
- **Personal assistant agents:** Plan and execute tasks on behalf of users *(e.g., travel planning, scheduling, information gathering)*.
- **Healthcare agents:** Support healthcare-related tasks *(e.g., summarising patient information, assisting with clinical documentation)*.
- **Financial agents:** Support financial tasks and decisions *(e.g., analyse financial information, prepare reports)*.

From Using an Agent to Engineering an Agent

Two different roles for agents:

- **Using an agent** *(e.g., Claude Code helping a developer modify and test a codebase)*

- **Engineering software that is itself an agent** *(e.g., building a customer-support agent with prompts, context, memory and tools).*

Agent = Model + Harness

- **LLM:** Provides the core language/reasoning capability.

- **Agent harness:** Software around the LLM that turns it into an operational agent by providing and managing:

  - **Instructions & context** — what the agent should do and what information it has.
  - **Memory** — information retained/retrieved across interactions.
  - **Tools/skills** — capabilities the agent can invoke to act.
  - **Orchestration** — controls the execution flow and interactions.
  - **Guardrails** — constrain actions and behaviour.
  - **Monitoring & evaluation** — observe and assess execution and results.
  - ….

Agent = Model + Instructions/Context + Memory + Tools/Skills + Orchestration + Guardrails + Evaluation + …

Quiz 3: True / False statements

1. An LLM that generates a response to a user is necessarily an AI agent.
2. An agent may use tools such as APIs, applications, search, or code execution to perform actions.
3. An agent can observe the results of its actions and use them to determine what to do next.
4. Memory can allow an agent to retain or retrieve relevant information across interactions.
5. Whereas in a coding assistant the developer typically drives the work, in a coding agent the agent can perform a sequence of actions toward completing a development task.
6. The LLM alone provides all the capabilities needed for an operational agent.
7. An agent harness can manage instructions/context, memory, tools, orchestration, guardrails, and evaluation around an LLM.

Quiz 3: Answers

1 False; 2 True; 3 True; 4 True; 5 True; 6 True; 7 False.

Quiz 4

Task: Label each example with the most appropriate concept: LLM (L), Agent (A), Agent Harness (H), Coding Assistant (CA), Coding Agent (CG)

1. Generates an answer from the provided prompt/context without independently carrying out actions.
2. Uses an LLM together with other capabilities to pursue a goal and perform actions.
3. Provides the infrastructure around an LLM for managing tools, context, orchestration, guardrails, and other agent capabilities.
4. Helps a developer write or understand code while the developer drives the development process.
5. Can work toward a software-development goal by carrying out a sequence of actions with greater autonomy.
6. Claude Code, Codex, and Gemini CLI are examples of this type of system.

Quiz 4: Answers

L, A, H, CA, CG, CG.

What Does This Mean for Computing Education?

Learning to use, build and engineer AI agents

- **Understand LLMs / LLM agents:** Understand their architecture and mechanisms—reasoning/planning, context, memory, tools and interaction.
- **Use AI in software development / and other tasks:** Use coding assistants / AI-native environments and coding agents to develop software.
- **Develop LLM agents:** Design and implement agents using prompts/instructions, context, memory, tools/skills and orchestration.
- **Evaluate / safeguard LLM agents:** Test and evaluate their behaviour, monitor their operation, apply guardrails, and perform verification and validation.

Retain foundations: Programming, CS/SE and AI/ML knowledge remain necessary to understand, evaluate and correct what agents produce.

From Software 1.0 to Software 3.0 (Karpathy Taxonomy)

- Code → Data/Learning → Natural Language + LLM

	Software 1.0	Software 2.0	Software 3.0
Intuition	Developer codes the rules (program)	Scientist provides data; the model learns the rules	User describes tasks; the LLM interprets and acts
What drives behaviour?	Human-written code	Learned model	LLM + natural-language instructions
Example	Traditional program	ML classifier	LLM agent

Ref: Source: Adapted from Andrej Karpathy, “Software Is Changing (Again),” AI Startup School, Y Combinator, June 2025.

Progress Across Software Paradigms

Software 1.0 (Traditional software)

- **Progress:** Shipping new features and fixing bugs; each sprint delivers code changes against a specification.
- **Typical iteration:** Modify code to implement a feature or fix a defect.
- **Example:** Add a password-reset feature and verify that the relevant tests pass.

Software 2.0 (ML models)

- **Progress:** Improving model quality on a specific prediction task.
- **Typical iteration:** Change model features, hyperparameters, training data/labels, …
- **Example:** Improve an HR intent classifier from 82% to 86% accuracy without unacceptable latency.

Software 3.0 (GenAI applications / agents)

- **Progress:** Improving overall agent/application performance across relevant quality dimensions.
- **Typical iteration:** Revise the prompt, switch model, change tools, orchestration, retrieval, guardrails, …
- **Example:** Improve a legal-proceedings summarizer so that its summaries are more accurate and complete.

Ref: Source: Adapted from ideas presented by Fabio Casati at ICSOC 2025.

Quality and Evaluation Across Software Paradigms

Software 1.0 (Traditional Software)

- **Quality:** Correct functionality + quality attributes (e.g. performance, usability, security)
- **Evaluation:** Unit, integration, system, acceptance, performance, security tests
- **Key characteristic:** Expected results are largely known. Tests can determine pass/fail

Software 2.0 (ML Models)

- **Quality:** Performance on a well-defined prediction task
- **Evaluation:** Evaluation datasets/benchmarks + statistical metrics (accuracy, precision, recall)
- **Key characteristic:** Model quality is measured by how well it performs across a set of examples, rather than requiring correct output for every test case

Software 3.0 (GenAI Applications / Agents)

- **Quality:** Multiple dimensions (task success, accuracy, completeness, safety, latency, execution cost)
- **Evaluation:** Benchmarks/evaluation datasets + metrics + human and/or LLM judges, …
- **Key characteristic:** Scorecards combining multiple quality dimensions (e.g., task success, accuracy, completeness, safety, latency, execution cost, leaderboard)

Quiz 5

Task: Label each statement with the most appropriate paradigm: Software **1.0 (S1),** Software **2.0 (S2),** Software **3.0 (S3)**

1. Behaviour is primarily defined through human-written code.
2. Behaviour is learned from training data.
3. Natural-language instructions and an LLM play a central role in determining system behaviour.
4. Improvement may involve changing training data, model features.
5. Improvement may involve changing prompts, models, tools, retrieval, orchestration or guardrails.
6. Quality is often evaluated using conventional tests with largely known expected results.
7. Evaluating the system may require assessing task success, tool use, safety and efficiency separately.

Quiz 5: Answers

1. S1; 2. S2; 3. S3; 4. S2; 5. S3; 6. S1; 7. S3

From Vibe Coding to Production AI

- **Vibe coding:** Build and modify an AI application mainly through intuition—change prompts/code, inspect a few outputs, and iterate based on what looks good.
- **Vibe coding is useful for prototyping:** Quickly shows whether an idea can work and produces the initial result.
- **Problem in production:** Checking 5–10 outputs does not show how the system behaves across the range of real inputs.
- **For agents:** Multiple steps, tool calls and generated outputs create more opportunities for failures that manual spot-checking can miss.
- **Need:** Move from *"it looks good"* to *"systematic evaluation"* of whether the system is actually good.

Multi-Dimensional Agent Evaluation

- **Traditional/TDD:** Tests typically check expected behaviour → **PASS / FAIL**
- **Agent evals:** One agent execution is assessed across **multiple quality dimensions**
- **Task success:** Did the agent achieve the requested goal?
- **Reasoning/planning:** Did it determine an appropriate sequence of steps?
- **Tool use:** Did it select and call the appropriate tools correctly?
- **Safety/compliance:** Did its actions respect constraints (e.g., business policies and regulations)?
- **Efficiency:** Did it complete the task with acceptable latency and token cost?

Evals as a Driver of Progress

Change → Evaluate → Compare → Analyse → Refine → Repeat

- **Change:** Modify the agent/application *(e.g., change the tool-selection prompt)*
- **Evaluate:** Run the evals *(e.g., run the same tool-calling test cases)*
- **Compare:** Compare with the previous version *(e.g., tool-selection success improves, but some cases still fail)*
- **Analyse:** Identify where/why it failed *(e.g., failures occur when tools have similar descriptions)*
- **Refine:** Use the findings for the next change *(e.g., improve tool descriptions and re-evaluate)*

Quiz 5: True / False statements

1. Inspecting a few good outputs is sufficient evidence that an LLM application is ready for production.

2. The same agent given the same task can behave differently across different runs.

3. Evaluating an agent only on whether it eventually completes the task can miss problems in areas such as tool use, safety and efficiency.

4. Agent evaluations must always be performed by a human evaluator.

5. After changing an agent, running the same evaluation suite can help determine whether the change improved performance or introduced regressions.

Quiz 5: Answers

1. False; 2. True; 3. True; 4. False; 5. True.

Key desirable Skills for the AI-Native Software Engineer (1)

From **Writing/developing software artifacts** to **Specifying + Directing + Understanding + Evaluating + Governing & Taking Responsibility**

- **Specify:** Define intent, requirements, constraints and checkable specifications that guide agents *(e.g., required functionality, APIs that may be used, performance constraints)*.

- **Direct:** Break work into tasks and guide/coordinate agents across various activities *(e.g., ask an agent to implement a feature, then test and fix it)*.

- **Evaluate:** Check agent-generated artefacts and results, identify failures *(e.g., review generated code, run tests, check whether requirements are satisfied)*

Refs: Stack Overflow Developer Survey 2025; Kang, Ray & Roychoudhury, *Skills for the Future Software Profession: Beyond Agentic AI!*,

Key desirable Skills for the AI-Native Software Engineer (2)

From **Writing/developing software artifacts** to **Specifying + Directing + Evaluating + Governing & Taking Responsibility**

- **Specify, Direct, Evaluate**
- **Maintain software understanding:** Review and understand what agents generate or modify rather than accepting changes blindly *(e.g., inspect code diffs and require the agent to explain what it changed and why).*
- **Govern & take responsibility:** Define what agents are allowed to do, put safeguards in place to prevent unsafe or unauthorized actions *(e.g., an agent cannot deploy code without approval)*, and ensure humans remain responsible for important decisions *(e.g., a developer approves the final deployment).*
- ….

Refs: Stack Overflow Developer Survey 2025; Kang, Ray & Roychoudhury, *Skills for the Future Software Profession: Beyond Agentic AI!*

Quiz 6: Specify or Direct?

Task: Label each activity as Specifying (S) or Directing (D).

1. Define that cancellation is allowed only up to 24 hours before departure.

2. Ask one agent to implement the feature and another to generate tests.

3. Define what the system should do when a user tries to cancel a booking less than 24 hours before departure.

4. Ask the coding agent to fix failures identified by the testing agent.

Quiz 6: Answer

1-S, 2-D, 3-S, 4-D

Quiz 7: What Should the Agent Be Allowed to Do?

A coding agent can:

1. Read the repository
2. Modify source code
3. Run tests
4. Change actual student records (e.g., change a student's module registration)
5. Deploy directly to production

Question: Which actions could be performed autonomously, and which should require human approval?

Quiz 7: Discussion

There is no single correct answer. Consider aspects such as:

- **Potential impact:** What happens if the action is wrong?
- **Reversibility:** Can the action and its consequences easily be undone?
- **Access & permissions:** Does it involve sensitive data, systems or privileged operations?
- **Human oversight:** Is human review or approval appropriate before execution?
- ….

General principle: Higher-impact, irreversible, privileged or difficult-to-verify actions should normally have stronger controls and human oversight.

Quiz 8: Specify

An LLM agent is being developed to answer questions about DCU modules.

Task: Which is a clearer specification of what the agent should do?

- **A.** Help DCU students.
- **B.** Answer students' questions about module schedules, assessments and teaching activities using official module information.

Quiz 8: Answer

B.

Quiz 9: Direct

You ask an agent to research a topic and prepare a short report. It produces a report that is too broad and does not address the required questions.

Task: As the person directing the agent, what should you do?

- **A.** Accept the report because the agent completed the task.
- **B.** Give the agent additional guidance about what needs to be addressed and ask it to revise the report.
- **C.** Give the agent more autonomy.

Quiz 9: Answer

B.

Quiz 10: Govern

An agent can:

- Search public university information.
- Read a student's personal record.
- Change a student's module registration.

Task: Should the agent be allowed to perform all these actions autonomously?

Quiz 10: Example Answer

- Search public information.
    - Yes, generally low risk.
- Read a student's record.
    - Can be done automatically when access is necessary for the task and the agent has appropriate permission.
- Change module registration.
    - Should not be done autonomously; require explicit approval before making the change.

Do We Still Need to Learn Programming and SE?

AI can increasingly generate code. Does a future software engineer still need to learn programming?

- **One view:** Natural-language programming reduces the need to master syntax and manually write every line of code.
- **Counterargument:** Without programming/SE knowledge, engineers may be unable to understand, inspect, debug or even recognise when AI-generated software is wrong.
- **Programming remains a way to learn foundations:** computational thinking, algorithms, abstraction, program behaviour and systematic problem solving.
- **The emphasis changes:** less focus on memorising syntax. More focus on understanding, reasoning about and evaluating software.

Refs: Mezini, *Who Says Programming Is Dead?*, 2025; Abrahão et al., *Software Engineering by and for Humans in an AI Era*, 2025.

Learning Software Engineering with AI

- **Use AI to accelerate learning:** Explore unfamiliar technologies, explain code, compare solutions and obtain feedback.
- **Do not outsource understanding:** Engineers still need to explain why a solution works and recognise when it does not.
- **Learn through debugging:** Investigating failures develops understanding of behaviour, dependencies and assumptions.
- **Develop strong engineering foundations:** Students need computing and software engineering knowledge to evaluate, debug and correct AI-generated software.

Foundations Still Matter

Traditional CS/SE foundations + AI/ML foundations → basis for effective AI-native engineering

- **Computing foundations:** Programming, algorithms, data structures, abstraction, systems, databases, networks and security.
- **Software-engineering foundations:** Requirements, architecture/design, testing/V&V, maintenance and evolution.
- **AI/ML foundations:** Models, data, training/inference, probabilistic behaviour, limitations and failure modes.
- **Domain knowledge:** Understand the application context sufficiently to judge whether AI-generated solutions are appropriate.
- **Foundations enable judgement:** They provide the knowledge needed to question, evaluate and correct AI-generated solutions.

Ref: Kang, Ray & Roychoudhury, *Skills for the Future Software Profession: Beyond Agentic AI!*

Learning AI-Native Software Engineering by Doing

Foundations + AI-native skills practised together through substantial projects

- **Use AI across the lifecycle:** Requirements, design, coding, testing, debugging, deployment and maintenance.
- **Practise AI-native skills:** Prompt/context engineering, agents/tools, harnesses/guardrails, evals and governance.
- **Experience failures:** Diagnose incorrect code, misunderstood requirements, security problems, etc.
- **Build complete systems:** Integrate AI-generated and traditional software rather than only completing isolated exercises.
- **Maintain understanding:** Students should be able to explain the architecture, important decisions and why the resulting system can be trusted.

Refs: Abrahão et al., *Software Engineering by and for Humans in an AI Era*, 2025.

Preparing Students for the AI-Native Job Market

Foundations + AI/ML knowledge + AI-native skills + project experience → evidence of engineering capability

- **Demonstrate foundations:** Show that AI use is backed by genuine computing and engineering knowledge.
- **Demonstrate AI-native skills:** Direct, evaluate, debug and control AI-generated work.
- **Show evidence through projects:** Working systems, architecture, tests/evals and documented engineering decisions.
- **Demonstrate failure handling:** Show how AI errors were detected, diagnosed and corrected.
- **Build experience before graduation:** Project-based learning provides students with practical evidence of engineering capability.
- **Prepare for technical interviews:** We will curate selected publicly available interview questions related to topics covered in CSC1202.

Summary

- **LLMs:** General-purpose models that can support many tasks through prompting, context, RAG and adaptation.
- **LLM-based agents:** Extend LLMs with capabilities such as tools, memory, planning and orchestration.
- **Agent harness:** Provides the infrastructure needed to build and operate agent systems.
- **Software 3.0:** Software engineering increasingly involves LLMs, natural-language instructions and agent components.
- **AI-native engineering:** Engineers need to specify, direct, govern and evaluate AI-generated work while maintaining understanding and responsibility.